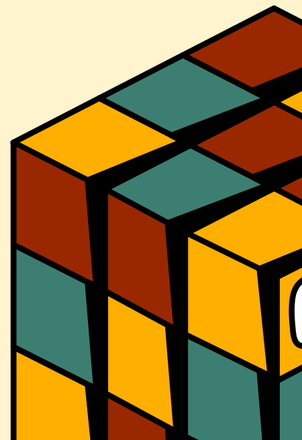
Creado por Daniel Quirós Cordero

Busca Minas

PROYECTO PORTAFOLIO POO

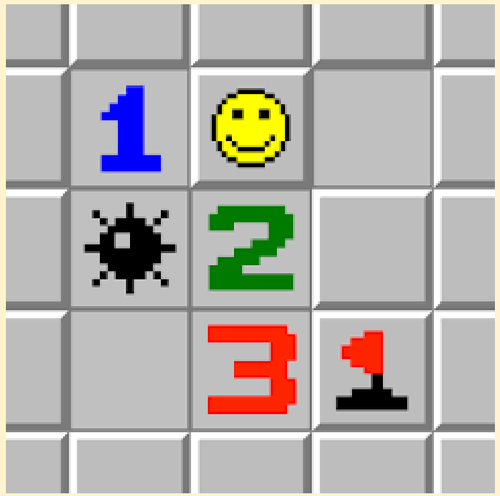
25 Octubre, 2025



¿Por qué Busca minas?

Justificacion y contexto

- **Justificación:** Sus componentes (tablero, celdas, jugador) se mapean naturalmente a clases. Permite demostrar estados, comportamientos polimórficos y reglas de negocio claras.
- **Contexto:** El jugador configura el tablero, revela celdas y marca banderas. El objetivo es descubrir todas las celdas seguras sin detonar una mina.



```
1. Revelar celda
2. Marcar bandera
Opcion: 2
Fila: 2
Columna: 1
- - - -
- - - -
- F 2 1
- - 1 0
```

Flujo del juego

Configuración

Se definen filas, columnas y minas. Los datos se validan mediante ConfiguracionJuego.

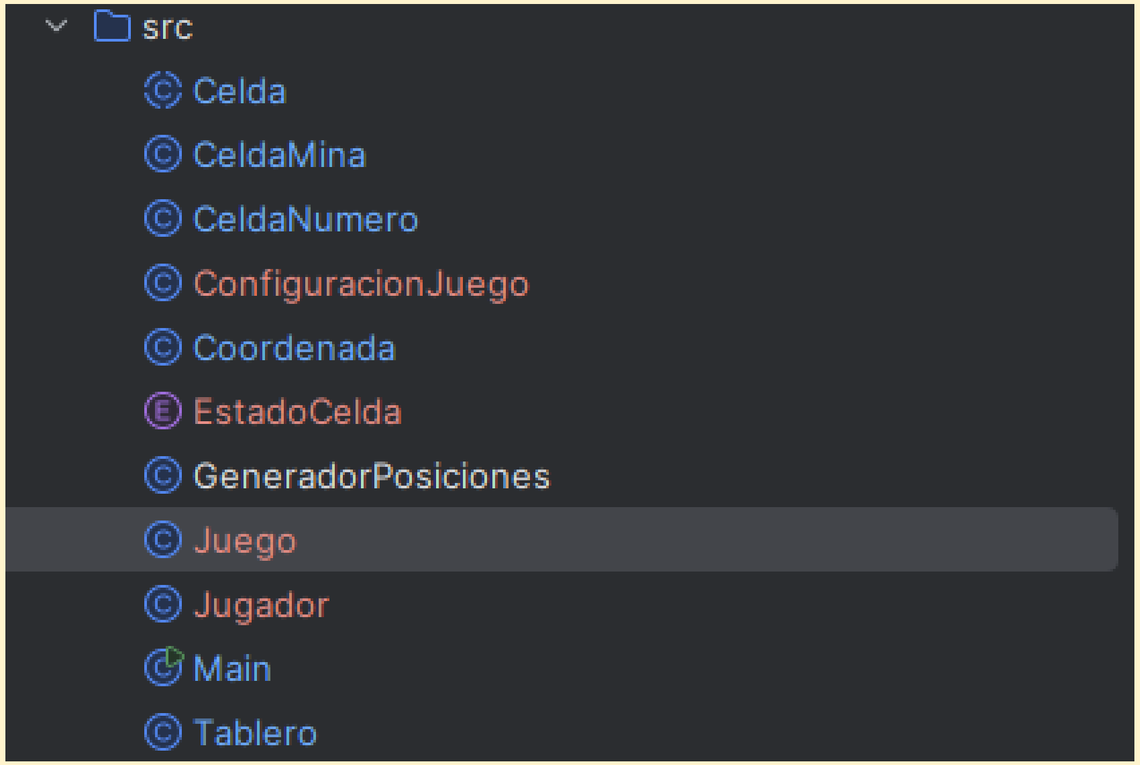
Interacción

El jugador revela celdas (se activa la cascada automática si hay 0 minas) o marca banderas.

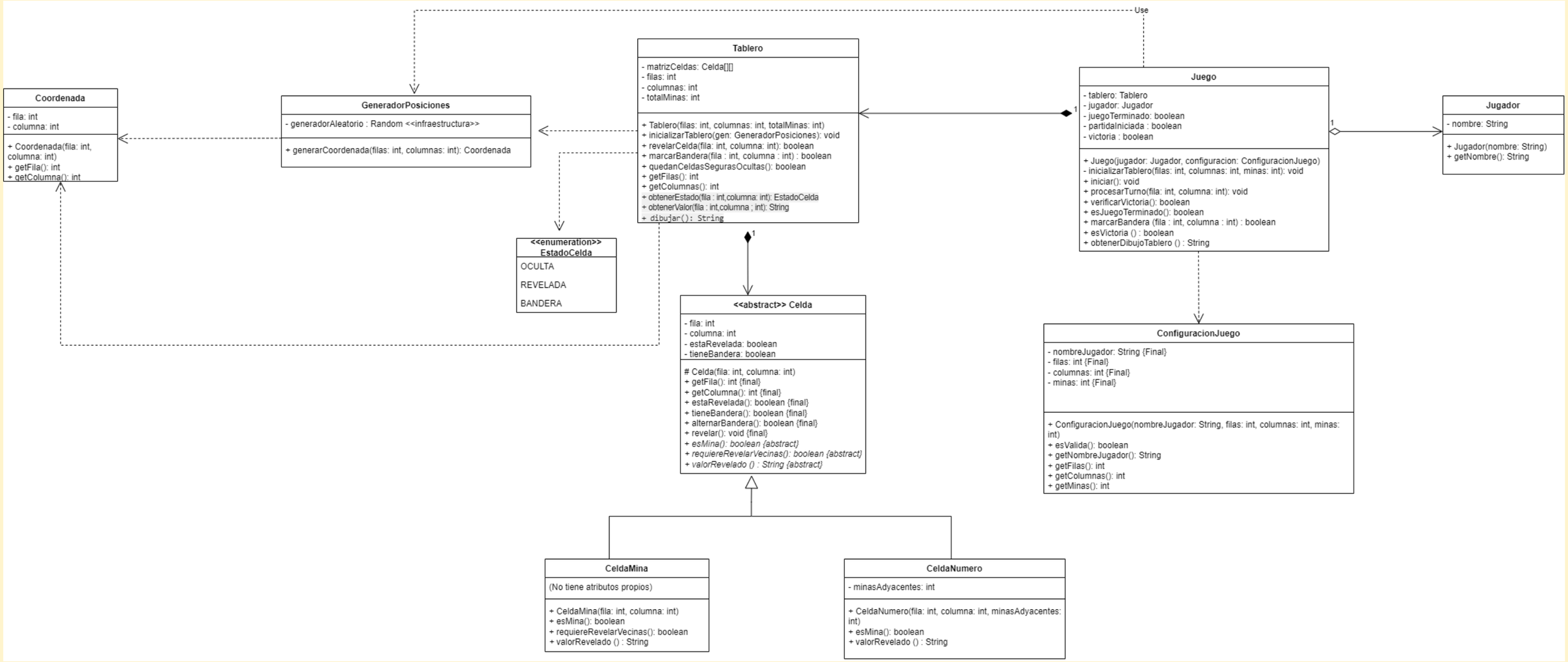
Resolución

Victoria (celdas seguras agotadas) o Derrota (mina revelada).

Estructura de clases



Arquitectura del sistema



Abstracción

Abstracción

```
public abstract boolean esMina();  2 usages  2 implementations  dquiroscenfo97
public abstract boolean requiereRevelarVecinas();  1 usage  2 implementations  new *
public abstract String valorRevelado();  1 usage  2 implementations  new *
```

Polimorfismo

```
@Override  2 usages  dquiroscenfo97
public boolean esMina(){
    return true;
}

@Override  1 usage  new *
public boolean requiereRevelarVecinas(){
    return false;
}

@Override  1 usage  new *
public String valorRevelado(){
    return "*";
}
```

- La clase Celda es abstracta. No existe una "celda genérica", solo minas o números.
- Esto permite que el Tablero trate a todas las celdas de forma uniforme sin conocer su tipo específico.

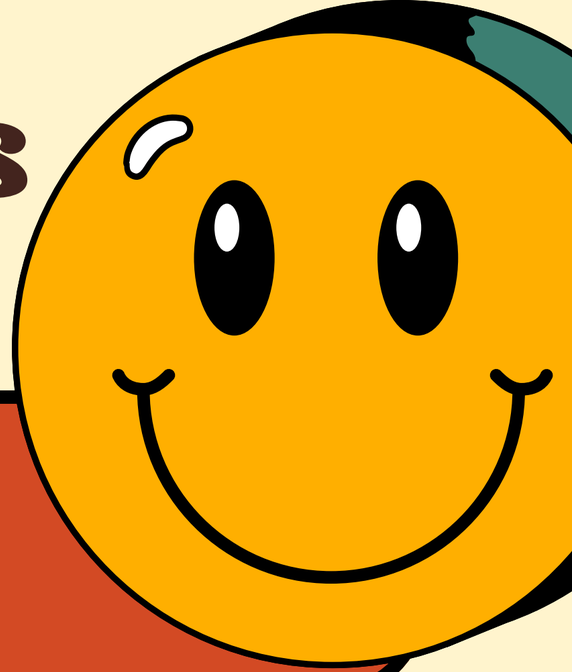
Encapsulamiento

```java
//Atributos
private Celda[][] matrizCeldas;
private int filas;  10 usages
private int columnas;  10 usages
private int totalMinas;  2 usages
```

```java
public EstadoCelda obtenerEstado(int fila, int columna) {  1 usage  new *
    if (fila < 0 || fila >= filas || columna < 0 || columna >= columnas) {
        return EstadoCelda.OCULTA;
    }
    Celda celda = matrizCeldas[fila][columna];
    if (celda == null) {
        return EstadoCelda.OCULTA;
    }
    if (celda.estaRevelada()) {
        return EstadoCelda.REVELADA;
    }
    if (celda.tieneBandera()) {
        return EstadoCelda.BANDERA;
    }
    return EstadoCelda.OCULTA;
}
```

- El estado interno está protegido. Tablero nunca expone su matriz de celdas.
- Solo devuelve datos primitivos (EstadoCelda, String) para que la vista no pueda mutar el modelo.

Relaciones entre objetos



Composición

- El tablero crea y controla el ciclo de vida de las celdas. El juego crea y controla el tablero.

Agregación

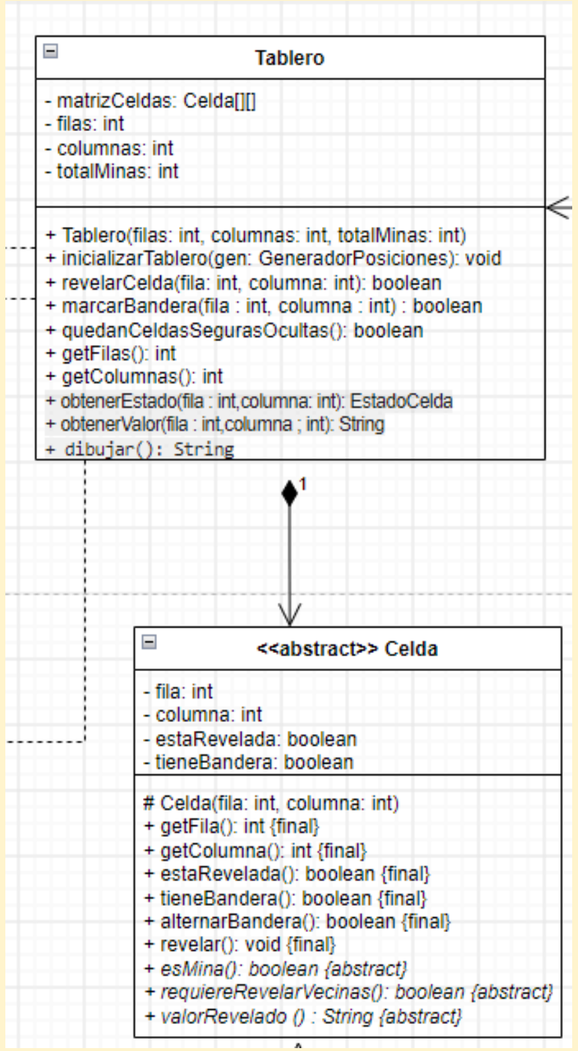
- El jugador existe antes y después de la partida.

Improved Customer Engagement:

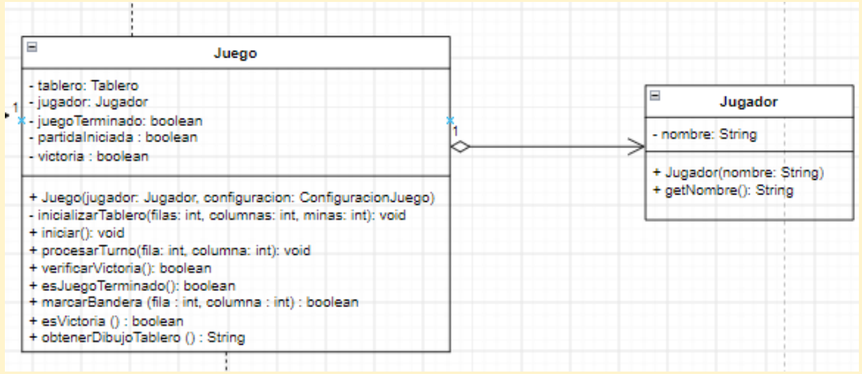
- El juego usa temporalmente la clase ConfiguracionJuego para inicializar parámetros validados.

Ejemplos UML

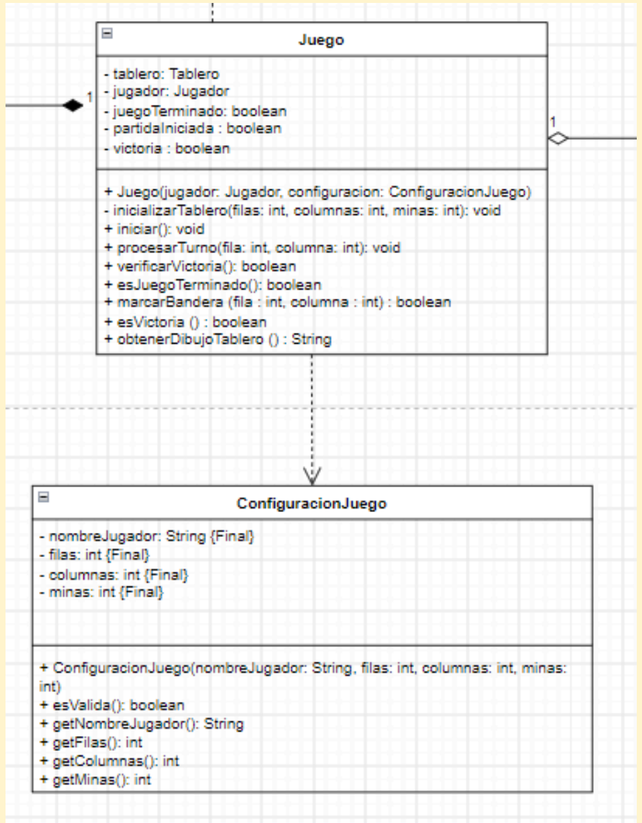
Composición



Agregación

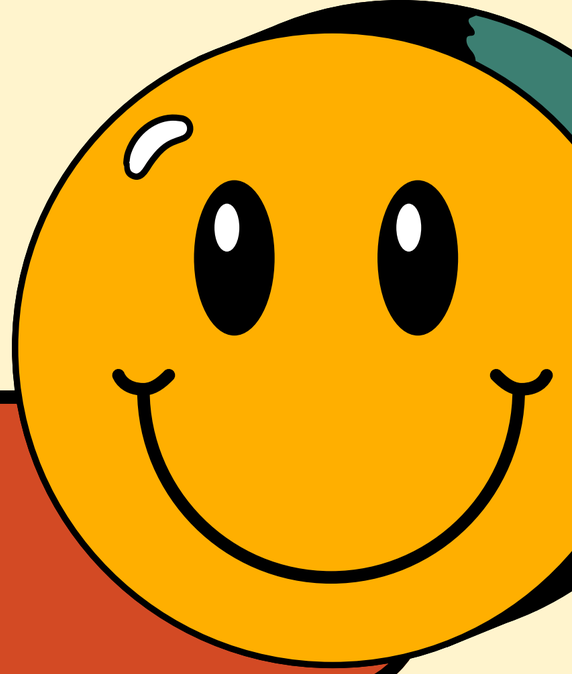


Dependencia



Flujo del programa

Inicio y config



Main

- Lee nombre, filas, columnas y minas desde la consola.

ConfiguracionJuego

- Crea un objeto inmutable con los parámetros.
- Ejecuta esValida(): verifica que las minas sean mayores a 0 y menores que el total de celdas.

Juego se ensambla

- Recibe el Jugador y la ConfiguracionJuego validada.
- Crea internamente el Tablero y delega la colocación de minas al GeneradorPosiciones.

Flujo de juego y resolución

El jugador elije: Revelar celda o marcar bandera

Juego delega la acción al Tablero.

Tablero consulta la celda específica:

- Si es **bandera**: alterna el estado.
- Si es **revelar**:
  Si es mina → juegoTerminado = true, victoria = false.
  Si es número con 0 minas → dispara cascada recursiva.
  Si es número con minas → revela solo esa celda.

Condición de fin

Condición de fin:

- **Victoria:** Tablero.quedanCeldasSegurasOcultas() devuelve false (no quedan celdas seguras por revelar).
- **Derrota:** Se reveló una mina.

Código destacado

inicializarTablero()

```java
public void inicializarTablero(GeneradorPosiciones generador){  1 usage  dquiroscenfo97 *
    boolean[][] hayMina = new boolean [filas][columnas];
    int minasColocadas = 0;
    while(minasColocadas < totalMinas){
        Coordenada coordenadaGenerada = generador.generarCoordenada(filas, columnas);
        while(hayMina[coordenadaGenerada.getFila()][coordenadaGenerada.getColumna()]){
            coordenadaGenerada = generador.generarCoordenada(filas, columnas);
        }
        hayMina[coordenadaGenerada.getFila()][coordenadaGenerada.getColumna()] = true;
        minasColocadas ++;
    }

    for(int i = 0; i < matrizCeldas.length; i++){
        for(int j = 0; j < matrizCeldas[i].length; j++){
            if(hayMina[i][j]){
                matrizCeldas[i][j] = new CeldaMina(i,j);
            }else{
                int minasAlrededor = 0;
                for(int desplazamientoFila = -1; desplazamientoFila <= 1; desplazamientoFila ++){
                    for(int desplazamientoColumna = -1; desplazamientoColumna <= 1; desplazamientoColumna ++){
                        if(desplazamientoFila != 0 || desplazamientoColumna != 0){
                            int filaVecina = i + desplazamientoFila;
                            int columnaVecina = j + desplazamientoColumna;
                            if(filaVecina >= 0 && filaVecina < hayMina.length &&
                            columnaVecina >= 0 && columnaVecina < hayMina[0].length){
                                if(hayMina[filaVecina][columnaVecina]){
                                    minasAlrededor ++;
                                }
                            }
                        }
                    }
                }
                matrizCeldas[i][j] = new CeldaNumero(i,j,minasAlrededor);
            }

        }
    }
}
```

Uso de la matriz de booleanos (hayMina) en inicializarTablero ()

La matriz boolean[][] hayMina actúa como una estructura de datos auxiliar y temporal para la fase de colocación de minas. Su propósito es separar la lógica de ubicación aleatoria de la creación de los objetos (CeldaMina y CeldaNumero).

Inicialización: Al crearse, Java inicializa todos sus valores en false por defecto.

Colocación y validación de colisiones: El bucle while genera coordenadas aleatorias mediante GeneradorPosiciones. El bucle while interno actúa como un mecanismo de rechazo: si hayMina[fila][columna] ya es true, significa que esa posición ya fue ocupada, por lo que se genera una nueva coordenada hasta encontrar un espacio vacío.

Marcado: Una vez hallada una posición válida, se marca como true y se incrementa el contador.

Esta matriz nunca se expone como atributo de la clase Tablero; es una variable local del método, lo que refuerza el encapsulamiento al ocultar los detalles internos de la inicialización (Salas Cerdas, 2026, 5.1 Relaciones entre objetos).

Recorrido de la matriz

Una vez definido el mapa lógico de minas (hayMina), el método recorre la matriz principal matrizCeldas para instanciar los objetos concretos.

- **Recorrido estructural:** Se utilizan dos bucles for anidados que iteran sobre matrizCeldas.length (filas) y matrizCeldas[i].length (columnas).
  - **Instanciación polimórfica:**Si hayMina[i][j] es true, se instancia new CeldaMina(i, j).
  - Si es false, se debe calcular el número de minas vecinas para instanciar new CeldaNumero(i, j, minasAlrededor).
- **Cálculo de adyacentes (Desplazamiento):** Para contar las minas alrededor de una celda segura, se utilizan dos bucles anidados con variables de desplazamiento (df y dc) que van de -1 a 1. Esto genera una "ventana" de 3x3 centrada en la celda actual.
  - Se omite el centro (df == 0 && dc == 0) porque es la celda actual.
  - **Validación de límites:** Antes de consultar hayMina, se verifica que las coordenadas vecinas (filaVecina, columnaVecina) estén dentro de los límites de la matriz (>= 0 y < length). Esto previene la excepción ArrayIndexOutOfBoundsException.
  - Si la vecina está dentro de los límites y hayMina es true, se incrementa el contador minasAlrededor.

revelarCelda()

```java
public boolean revelarCelda(int fila, int columna){  2 usages  new *
    if(fila < 0 || fila >= this.filas || columna < 0 || columna >= this.columnas){
        return false;
    }
    Celda posicionActual = matrizCeldas[fila][columna];

    if(posicionActual == null){
        return false;
    }

    if(posicionActual.estaRevelada()){
        return false;
    }

    if(posicionActual.tieneBandera()){
        return false;
    }
    posicionActual.revelar();

    if(posicionActual.estaRevelada() && posicionActual.esMina()){
        return true;
    }
    if(posicionActual.estaRevelada() && posicionActual.requiereRevelarVecinas()){
        for(int df = -1; df <= 1; df ++){
            for(int dc = -1; dc <= 1; dc ++){
                if(df == 0 && dc == 0){
                    continue;
                }
                int vecinaFila = fila + df;
                int vecinaColumna = columna + dc;
                if (vecinaFila >= 0 && vecinaFila < filas &&
                    vecinaColumna >= 0 && vecinaColumna < columnas) {

                    if (revelarCelda(vecinaFila, vecinaColumna)) {
                        return true;
                    }
                }
            }
        }
    }
    return false;
```

Recursividad en revelar celda

La recursividad se utiliza para descubrir automáticamente las celdas vecinas cuando el jugador revela una celda que no tiene minas adyacentes (un 0). El método se llama a sí mismo de forma controlada.

Para que la recursividad sea segura y no colapse el programa, el método define casos base (condiciones de parada) al inicio:

1. **Límites: Si la coordenada está fuera del tablero, retorna false.**
2. **Estado nulo: Si la celda no existe, retorna false.**
3. **Estado revelado: Si la celda ya fue abierta, retorna false (evita bucles infinitos).**
4. **Bandera: Si la celda tiene bandera, retorna false (protege la celda marcada).**

El paso recursivo: Si la celda supera los casos base, se revela (posicionActual.revelar()). Luego, evalúa si debe expandirse consultando el método polimórfico requiereRevelarVecinas() (que solo devuelve true en CeldaNumero si tiene 0 minas).

Si se cumple, recorre sus 8 vecinas usando desplazamientos (df, dc), valida los límites nuevamente, y se invoca a sí misma: revelarCelda(vecinaFila, vecinaColumna).

Si en alguna profundidad de la recursividad se encuentra una mina, el método retorna true. Este valor booleano se propaga hacia arriba en la "pila de llamadas" (call stack), deteniendo la cascada y notificando al Juego que la partida ha terminado.